

Linguaggi di Programmazione

Modulo II

Nicolò Giuliani

Indice

1	Nomi e ambiente	2
1.1	Regole di scope	3
2	Memoria	4
2.1	Implementazione delle regole di scope	6
3	Controllo	7
3.1	Espressioni	7
3.2	Comandi	7
3.2.1	Iterazione	9
3.3	Ricorsione	9
4	Sottoprogrammi	10
4.1	Parametri	10
4.2	Downward funarg problem	11
4.3	Upward funarg problem	12
4.4	Eccezioni	13

1 Nomi e ambiente

L'uso di un **nome** serve ad indicare l'oggetto denotato.

```
const pi = 3.14;
int x;
void f() {...};
```

π , x , f sono nomi. Gli oggetti che denotano:

- la costante 3.14
- una variabile
- la definizione di f

Nei linguaggi i nomi sono **token** (identificatori). Il loro utilizzo serve per indicare l'**oggetto denotato**.

Definiamo **ambiente** ovvero l'insieme delle associazioni fra nomi e oggetti denotabile esistenti a run-time in uno specifico punto del programma ed in uno specifico momento dell'esecuzione. Attraverso la **dichiarazione** (meccanismo col quale si) si crea un'associazione in ambiente.

```
int x;
int f(){
    return 0;
}
type T = int;
```

Bisogna stare attenti come in diversi punti del programma lo stesso nome può essere utilizzato per oggetti diversi. Mentre attraverso **aliasing** possiamo avere nomi diversi per lo stesso oggetto (puntatori per riferimento, puntatori)

```
int *x;
x = (int *) malloc(sizeof(int))
*x = 5;
```

Nei linguaggi moderni l'ambiente è strutturato attraverso i **blocchi**. Ovvero regioni testuali con inizio e fine che può contenere dichiarazioni locali in quella regione.

```
{
int tmp = x;
x = y;
y = tmp;
}
```

Due blocchi si sovrappongono se un blocco sta dentro un altro blocco. La dichiarazione locale è libera per un blocco annidato, a meno che una nuova dichiarazione venga fatta nel sottoblocco. L'ambiente locale può essere suddiviso in

- ambiente locale: associazioni all'ingresso del blocco
- ambiente non locale: associazioni eritate da altri blocchi
- ambiente globale: parti di ambiente non locale relativo alle associazioni comuni (variabili globali, ecc)

La **vita** di un oggetto e un legame è:

- vita dell'oggetto più lunga

```
fn P(int X){...} //definiamo la funzione
int A; //definiamo la variabile
P(A); //creiamo il legame
```

Durante l'esecuzione di $P(A)$ esiste un legame tra X e l'oggetto che esisteva già prima e dopo l'esecuzione della funzione.

- vita dell'oggetto più breve (area di memoria de-allocata)

```
int *x, *y;
x = (int *) malloc(sizeof(int));
y = x;
free(x);
x=null;
```

Dopo la $free(x)$ non esiste più l'oggetto ma esiste ancora un legame per y (dangling reference)

1.1 Regole di scope

Come implementiamo la regola di visibilità?

Una dichiarazione locale è visibile in quel blocco e in tutti i blocchi annidati, a meno di nuove dichiarazioni con stesso nome.

```
{int x=10;
  void foo () {
    x++;
  }
  void fie () {
    int x = 0;
    foo();
  }
  fie();
}
```

Quale x incrementa $foo()$? Siamo in un caso di un riferimento non-locale, può essere risolto:

- Scope statico (c, java, rust, ecc): nel blocco che include sintatticamente $foo()$

$$x = 11$$

- Scope dinamico (Emacs Lisp, perl, ecc): nel blocco che è eseguito immediatamente prima di $foo()$

$$\begin{aligned} x_{\text{locale}} &= 1 \\ x &= 10 \end{aligned}$$

Scope statico

```
{ int x=0;
void pippo(int n){
    x = n+1;
}
pippo(3);
// incrementa x globale
write(x);
// stampa x globale = 4
{int x=0;
// ridifiniamo x locale
pippo(3);
// incrementa x globale
write(x);
// stampa x locale = 0
}
write(x);
// stampa x globale = 4
}
```

Output: 404

Scope dinamico

```
{ int x=0;
void pippo(int n){
    x = n+1;
}
pippo(3);
// incrementa x globale
write(x);
// stampa x globale = 4
{int x=0;
// settiamo x locale
pippo(3);
// incrementa x di blocco chiamante
write(x);
// stampa x locale = 4
}
write(x);
// stampa x globale = 4
}
```

Output: 444

Per l'allocazione di memoria guarda: Implementazione delle regole di scope.

2 Memoria

Come allochiamo la memoria?

- Statica: allocazione a tempo di compilazione.
Ovvero zone protette di memoria dove gli oggetti hanno indirizzi assoluti per tutta l'esecuzione. (variabili globali, costanti, ecc). Non permette la ricorsione, perche tiene SOLO le variabili locali per sottoprogrammi).

```
fn fatt(x){
    if x == 0 {
        return 1
    }
    else return x*fatt(x-1)
}
fatt(5):
    5*fatt(4)
fatt(4):
    4*fatt(3)    \\ sovrascrive il vecchio x
fatt(3):
    3*fatt(2)    \\ sovrascrive ancora
```

Quindi sovrascrivendo sempre x non possiamo mai avere un valore di ritorno valido per la ricorsione.

- Dinamica: allocazione a tempo di esecuzione.

- **Stack** (FIFO): Definiamo che ogni istanza di un sottoprogramma ha una porzione di memoria chiamato **frame stack** (record di attivazione o RdA), con le relative informazioni (anche indirizzo di ritorno di memoria).

Abbiamo un puntatore **SP** (stack pointer) che punta sempre il cima allo stack, e poi abbiamo anche **FP** (frame pointer) che indica la base del frame. Così possiamo accedere a tutte le variabili con

$$\text{indirizzo} = \text{FP} + \text{offset}$$

con FP calcolato a compile-time.

Perchè utilizziamo gli RdA e il link dinamico? Gli RdA non hanno tutti la stessa dimensione, dipende da paramatri, variabili locali, ecc, quindi il *pop()* non è così semplice. Allora attraverso il link dinamico abbiamo

	variabili locali		
	parametri		
	indirizzo ritorno		
	link_dinamico		\\ punta al frame precedente
			\\ che ci ha chiamato

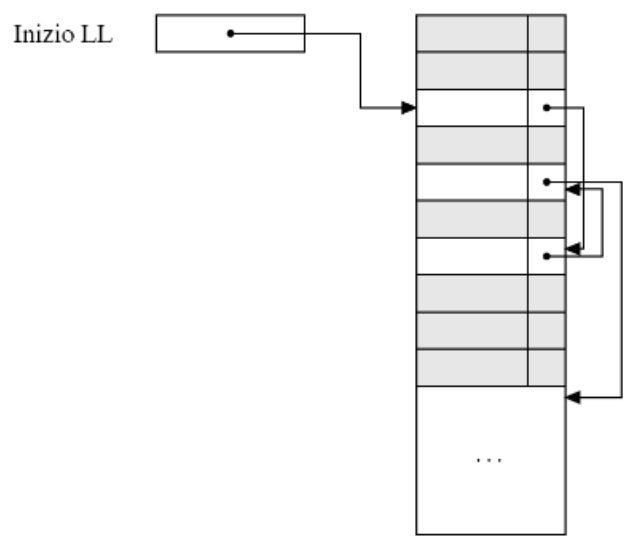
e quindi basta fare $\text{SP} = \text{link_dinamico}$.

- **Heap**: memoria nella quale i (sotto)blocchi possono essere allocati/deallocati dinamicamente arbitrariamente.

Questo ci serve per gli oggetti di dimensione dinamica come stringhe, liste, alberi, puntatori, ecc.

Abbiamo due modi di implementazione:

- Blocchi di memoria di dimensione fissa, li mettiamo nella **LL** (lista libera). Quando allochiamo shiftiamo il puntatore LL al primo blocco libero, mentre quando deallochiamo LL punta a quel blocco che punta al ex-primo blocco libero.



- Blocchi di memoria di dimensione variabile, partiamo con un unico blocco. Quando allochiamo prendiamo una parte del blocco di dimensione richiesta, mentre la deallocazione la restituisce alla LL. Bisogna però stare molto attenti alla frammentazione della memoria!

- * Frammentazione interna: allochiamo un blocco che però è più grande di quello richiesto (spreco di memoria)
- * Frammentazione esterna: abbiamo abbastanza spazi sparsi ma separati

Come gestiamo quindi la LL unica? Ad ogni allocazione possiamo fare:

- first fit: primo blocco abbastanza grande
- best fit: minor blocco abbastanza grande

Possiamo migliorare? Attraverso LL multiple (blocchi dimensione variabile). Però come ripartiamo i blocchi fra le LL?

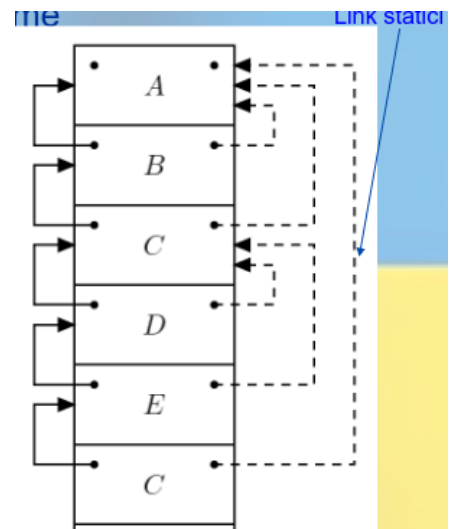
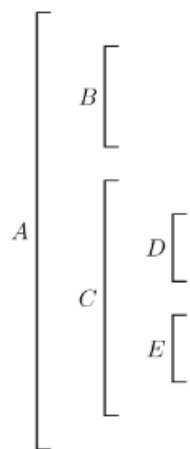
- statica
- dinamica

2.1 Implementazione delle regole di scope

- Scope statico

- Catena statica
Immaginiamo di Avere

main $\rightarrow$ A $\rightarrow$ B $\rightarrow$ C $\rightarrow$ D $\rightarrow$ E $\rightarrow$ C



Se un sottoprogramma è a livello k implica che la catena è lunga k .

Se $k = 0$ il chiamante punta SP alla funzione, se $k > 0$ il chiamante risale la catena k -volte. Ogni volta che incontriamo un nome ci segniamo se è locale con $h = 0$ o se era stato definito h -blocchi fa.

- Display

Possiamo ottimizzare il costo h per la lettura di un nome non locale attraverso un array con i RdA attivo definito ad RdA $i - 1$. Così possiamo trovare la posizione di un oggetto con $i - h$. Quando aumentiamo la catena $k + 1$ il chiamante e il processo non condividono più il $Display[k + 1]$.

Nella realtà il display è più costoso nella sequenza di chiamate e quindi è poco utilizzato.

- Scope dinamico

- A-list
Le associazioni sono memorizzate in una struttura apposita, manipolata come una pila.
- CRT (Tabella centrale dell'ambiente)
Una tabella mantiene tutti i nomi distinti del programma. Se i nomi sono noti il costo è costante, se no costo hash.

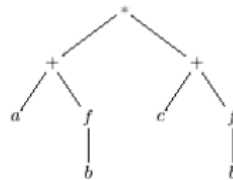
3 Controllo

Come strutturiamo il controllo del flusso di un linguaggio?

3.1 Espressioni

Entità sintattica che produce un valore o è indefinita. La sintassi può essere

- Infissa: $a + b$
Gli operatori aritmetici hanno precedenza su confronto che hanno a loro volta precedenza su logici. Le parentesi sono essenziali in alcuni casi. L'associatività dipende dal tipo di operando.
- Prefissa: $+ab$
Si valuta attraverso una pila, si legge il prossimo simbolo e lo si mette sulla pila, se abbiamo un operatore applichiamo l'operazione su i valori della pila prima del operatore e lo memorizziamo R . Eliminiamo operatore e operandi, memorizziamo R in pila.
- Postfissa: $ab+$
Sempre attraverso una pila ma dobbiamo contare gli operandi che vengono letti



Le espressioni possono essere rappresentate come alberi, a seconda del tipo di visita di albero abbiamo una delle 3 precedenti notazioni lineare. A partire dall'albero il compilatore produce il codice oggetto oppure l'interprete valuta l'espressione.

Ovviamente la valutazione può comportare dei risultati diversi (effetti collaterali)

$$(a+f(b)) * (c+f(b))$$

Se $f(b)$ modifica b allora il risultato da sinistra a destra è diverso di quello da destra a sinistra.

3.2 Comandi

Un comando è un' entità sintattica la cui valutazione non necessariamente restituisce un valore, ma può avere un effetto collaterale.

Sapendo che una **variabile** è un "contenitore" modificabile di valori con un nome. Attraverso il comando di assegnamento possiamo modificare il valore di una variabile.

```
x := 2
x = x+1
```

Il primo x è un l-value ovvero una locazione, mentre il secondo x è un r-value ossia un valore che può essere contenuto in una locazione. Anche la assegnazione comporta effetti collaterali (anche se non ritorna un valore sempre).

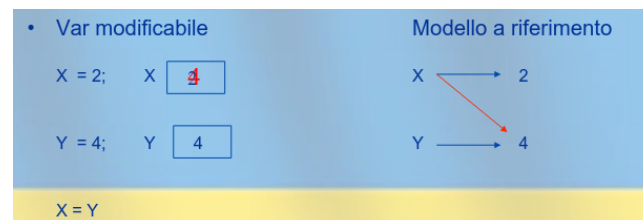
```
//C
x = 2; //ritorna 2
y = x = 2; //valido
```

Questo vale un po' per tutti i linguaggi imperativi. Mentre nei linguaggi funzionali (Haskell, ecc) assomiglia più al modello matematico, una variabile denota un valore ed è immutabile. I linguaggi logici danno la possibilità di modifica entro certi limiti.

Abbiamo anche il **modello a riferimento**, ovvero una variabile è riferimento ad un valore che ha un nome

```
x —> 2
```

Simile ai puntatori, ma i puntatori hanno le allocazioni esplicite.



Ambiente e memoria

: Due variabili diverse possono denotare lo stesso oggetto? Nomi $\rightarrow$ Valori non basta!
 Nei linguaggi imperativi abbiamo

- Valori Denotabili (quelli che possono dare i nomi)
- Valori Memorizzabili (quelli che si possono memorizzare)
- Valori Esprimibili (risultato della valutazione di una exp.)

Come gestiamo i **comandi per il controllo sequenza**?

Esplicitamente con ";", "blocchi" o "goto", ma non bastano per la *programmazione strutturata*. **Comandi condizionali** come "if case" o comandi iterativi "for while". Questi ultimi 2 comandi permettono di avere codice ben strutturato e non "spaghetti code". Questi permettono una maggiore leggibilità e anche ottimizzazione se compilato in modo astuto.

Come sappiamo Iterazione e ricorsione sono i due meccanismi che permettono di ottenere formalismi di calcolo Turing completi. Senza di essi avremmo automi a stati finiti.

3.2.1 Iterazione

Abbiamo due tipologie di iterazione

- indeterminata: cicli controllati logicamente, non noto a priori al avvio del ciclo (while, repeat, ecc). Questo è il vero potere delle MdT. Si implementano attraverso l'istruzione di salto condizionato della macchina fisica.
- determinata: cicli controllati numericamente, ovvero noto a priori (do, for, ecc)

3.3 Ricorsione

Modo alternativo all'iterazione per ottenere il potere espressivo delle MdT.

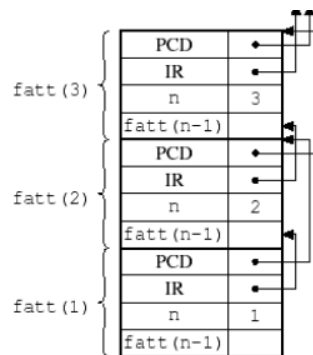
La definizione di una funzione ricorsiva è analoga alla definizione induttiva di una funzione. Ogni programma iterativo può essere riscritto in ricorsivo e viceversa.

In caso di implementazioni naive ricorsione meno efficiente di iterazione possiamo utilizzare optimizing compiler che produce codice efficiente o tail-recursion.

La ricorsione in coda (tail recursion): Una chiamata di g in f si dice "chiamata in coda" (o tail call) se f restituisce il valore restituito da g senza ulteriore computazione.

```
function tail_rec (n: integer): integer
  begin ... ; x:= tail_rec(n-1) end
function non_tail_rec (n: integer): integer
  begin ... ; x:= non_tail_rec(n-1); y:= g(x) end
```

Qui la chiamata ricorsiva è l'ultima operazione eseguita dalla funzione, la funzione ritorna direttamente il risultato della chiamata ricorsiva. Dopo $tail_rec(n-1)$ non c'è più nulla da fare. Non serve allocazione dinamica della memoria con pila: basta un unico RdA.



Possibile la generazione di codice tail-recursive usando continuation passing style (passare il resto come parametro), questo rende più efficiente il costo computazionale e di spazio.

```
int fibrc (int n, int res1, int res2){
  if (n == 0)
    return res2;
  else
    if (n == 1)
      return res2;
    else
      return fibrc(n-1, res2, res1+res2);
}
```

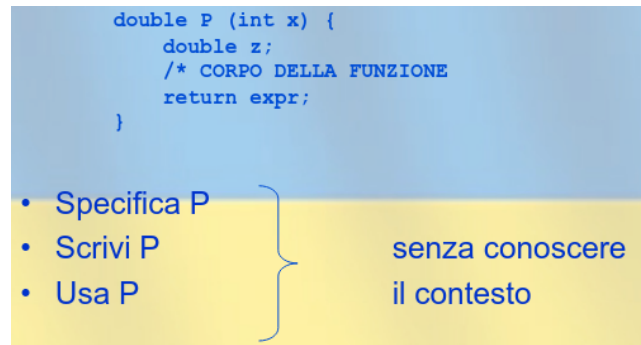
Costo lineare di tempo n e spazio costante (1 RdA).

4 Sottoprogrammi

I LP sono essi stessi astrazioni del calcolatore sottostante.

LP alto livello $\iff$ maggiore astrazione

- Astrazione sul controllo: fornisce astrazione funzionale al progetto (comunica con parametri, ambiente globale)



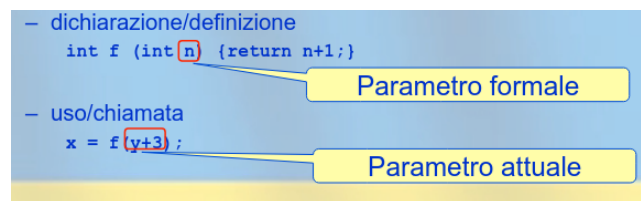
```
double P (int x) {  
    double z;  
    /* CORPO DELLA FUNZIONE  
    return expr;  
}
```

- Specifica P
- Scrivi P
- Usa P

senza conoscere il contesto

- Astrazione sui dati: implementazione dei dati/operazioni inaccessibile all'utente

4.1 Parametri



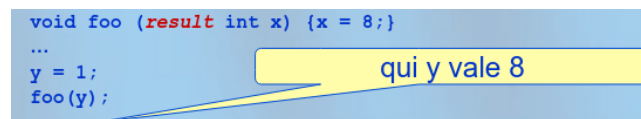
```
- dichiarazione/definizione  
int f (int n) {return n+1;}  
  
- uso/chiamata  
x = f(y+3);
```

Parametro formale

Parametro attuale

Come passiamo i parametri?

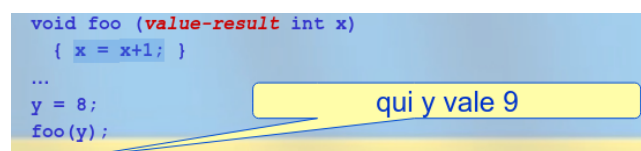
- per valore: il valore dell'attuale è assegnato al formale, che si comporta come una variabile locale
- per riferimento: è passato un riferimento (indirizzo) all'attuale; i riferimenti al formale sono riferimenti all'attuale (*aliasing*)
- per costante (read-only): nella procedura non è permessa la modifica del formale
- per risultato



```
void foo (result int x) {x = 8;}  
...  
y = 1;  
foo(y);
```

qui y vale 8

- per valore/risultato



```
void foo (value-result int x)  
{ x = x+1; }  
...  
y = 8;  
foo(y);
```

qui y vale 9

- per nome: una chiamata di P è la stessa cosa di eseguire il corpo di P dopo aver sostituito i parametri attuali al posto dei parametri formali

```

int y;
void fie (name int x){
    int y;
    x = x + 1; y = 0;
}
...
y = 1;
fie(y);

```

```

{ int y;
  y = y+1;
  y = 0; }

```

Può portare conflitti di nomi!

Alcune funzioni permettono di passare come parametro altre funzioni (puntatore al RdA all'interno della pila). O magari restituire delle funzioni (mantenere il RdA della funzione restituita, la pila non va più bene). Come gestire l'ambiente della funzione?

```

int x=4; int z=0;
int f (int y){
    return x*y;}
void g ( int h(int n) ) {
    int x;
    x = 7;
    z = h(3) + x ;
    end;}
...
{ int x = 5;
  g(f);
}

```

Quale x utilizziamo in f ?

- scope statico: la x esterna
- scope dinamico: ha senso sia la x del blocco di chiamata che la x interna

4.2 Downward funarg problem

Quando una procedura viene passata come parametro, si crea un riferimento tra un nome (par formale: h) e una procedura (par attuale: f).

Ovvero: Quale ambiente non locale si applica al momento dell'esecuzione di f (in quanto chiamata via h)?

- **Deep binding**: ambiente al momento della creazione del legame $h \rightarrow f$ (scope statico)
- **Shallow binding**: ambiente al momento della chiamata di f via h (forse scope dinamico)

```

int x=4; int z =0;
int f (int y){
    return x*y; }
void g (int h(int n)){
    int x ;

    x = 7;
    z = h(3) + x
}

...
{int x = 5;
  g(f);
}

```

- Scope statico:
 - Deep: x rossa (esterna)
 - Shallow: x rossa (esterna)
- Scope dinamico:
 - Deep: x azzurra (blocco di chiamata)
 - Shallow: x nera (blocco interno)

Passare una chiusura Passare dinamicamente sia il legame col codice della funzione, che il suo ambiente non locale, cioè una chiusura $\langle code, env \rangle$.

Quindi allochiamo (come sempre) RdA, prendiamo il puntatore di catena statica dalla chiusura.

Come li implementiamo?

- Scope statico
 - (SEMPRE) Deep: implementato con chiusure
 - Shallow: non si fa, ma non fa differenza con deep
- Scope dinamico
 - Deep: usa necessariamente qualche forma di chiusura per “congelare” uno scope da riattivare più tardi
 - Shallow: per accedere ad x risali la pila (A-list, CRT)

4.3 Upward funarg problem

Alcuni linguaggi (eg funzionali) permettono di restituire una funzione. Se la funzione ha variabili locali queste devono sopravvivere indipendentemente dalla struttura a pila: hanno vita indefinitamente lunga.

```

{int x = 1;
void->int F () {
    int g () {
        return x+1;
    }
    return g;
}
void->int gg = F();
int z = gg();
}

```

F ritorna una chiusura.

La pila (LIFO) non va più bene, come implementiamo la "pila" corretta?

- non deallocare esplicitamente
- record di attivazione sullo heap
- le catene statica e dinamica collegano i record
- invoca il garbage collector quando necessario

4.4 Eccezioni

Terminare parte di una computazione quando richiesto e gestire l'eccezione se sorta.

• La funzione **average** calcola la media di un vettore; se il vettore è vuoto, solleva eccezione

```
class EmptyExcp extends Throwable {int x=0;};

int average(int[] V) throws EmptyExcp() {
    if (length(V)==0) throw new EmptyExcp();
    else {int s=0; for (int i=0, i<length(V), i++) s=s+V[i];}
    return s/length(V);
};

try {
    ...
    average(W);
    ...
}
catch (EmptyExcp e) {write('Array_vuoto'); }
```

solleva l'eccezione

eccezione

intrappola e gestisce l'eccezione

gestore (handler) dell'eccezione

Il gestore è legato in modo statico al blocco di codice protetto, l'esecuzione del gestore rimpiazza la parte di blocco che doveva essere ancora eseguita.

Possiamo anche utilizzare le eccezioni per rendere più efficiente un codice:

Problema: Calcolare il prodotto tra tutti i nodi di un albero

- NON OTTIMIZZATA:

```
int mul(T tree){
    if (tree == null) return 1;
    return tree.value * mul(tree.L) * mul(tree.R);
}
```

- OTTIMIZZATA:

```
int mulAus(T tree){
    if (tree == null) return 1;
    if (tree.value == 0) {
        throw new Zero();
    }
    return tree.value * mul(tree.L) * mul(tree.R);
}
```

```

    }

    int mul(T tree){
        try{
            return mulAus(tree)
        } catch (Zero e){
            return 0;
        }
    }
}

```

Come implementiamo le eccezioni? Il compilatore genera una *tabella delle eccezioni* che, per ogni blocco protetto (`try`), contiene una coppia del tipo

$$\langle block_addr, handler_addr \rangle$$

(dove in realtà il blocco è identificato da un intervallo di indirizzi).

La tabella è ordinata staticamente rispetto a *block_addr*.

Quando viene sollevata un'eccezione, il runtime prende l'indirizzo corrente di esecuzione (Program Counter) e cerca nella tabella il blocco che lo contiene, tipicamente tramite *ricerca binaria*.

Una volta individuata la voce corrispondente, il controllo viene trasferito all'indirizzo del gestore (*handler_addr*), dopo aver effettuato lo *stack unwinding*.

Se il gestore rilancia l'eccezione, il processo viene ripetuto nel frame chiamante, continuando la ricerca di un gestore compatibile. Se non lo trova il programma finisce senza gestire l'eccezione.